1. Thread vs. Runnable

Both are used to define a task that runs in a separate thread, but they represent different design philosophies.

- **Thread (Class): When you extend the Thread class, your class *is* a thread.**
    - **Downside: Java doesn't support multiple inheritance. If you extend Thread, you cannot extend any other class (like a BaseService).**
- **Runnable (Interface): This represents a *task* that can be executed by a thread.**
    - **Upside: It separates the task logic from the execution mechanism. You can implement Runnable and still extend another class. This is the preferred approach in modern Java.**
    - **Example: You define a LogMigrationTask as a Runnable, and then pass it to an ExecutorService to run it.**

---

2. The synchronized Keyword

This is Java's primary tool for Mutual Exclusion (Mutex). It ensures that only one thread can execute a specific block of code at a time.

- **Instance Level: synchronized void method() locks the entire object instance. No other thread can call *any* synchronized method on that same instance.**
- **Static Level: synchronized static void method() locks the Class object. It affects all instances of that class.**
- **Block Level: synchronized(lockObject) { ... } is more granular. It only locks a specific piece of code. This is better for performance because it reduces the "locking overhead."**

---

3. Race Condition

A race condition occurs when two or more threads try to change shared data at the same time. The final result depends on the timing (the "race") of the threads.

- **The Classic Example (Check-then-Act): Imagine an Integer count = 0. Two threads try to do count++.**
    1. **Thread A reads count (0).**

2. **Thread B reads count (0).**
3. **Thread A increments to 1 and saves.**
4. **Thread B increments to 1 and saves.**

- **The Result: The count is 1, but it should have been 2. Data is lost.**
- **The Fix: Use synchronized, ReentrantLock, or Atomic Variables (like AtomicInteger) which use low-level CPU instructions to ensure the increment is a single, uninterruptible operation.**

---

4. ExecutorService

In production, you should never manually create threads using new Thread().start(). Creating threads is expensive for the OS. Instead, you use a Thread Pool via ExecutorService.

How it works:

1. **Work Queue: When you submit a task, it goes into a queue (usually a LinkedBlockingQueue).**
2. **Worker Threads: A fixed number of threads sit idle, waiting for tasks.**
3. **Execution: A thread pulls a task from the queue, executes it, and then returns to the pool to wait for the next one.**

Common Pool Types:

- **FixedThreadPool(n): Keeps exactly n threads alive. Good for predictable loads.**
- **CachedThreadPool: Creates new threads as needed and kills them when idle. Good for many short-lived tasks.**
- **ScheduledExecutorService: Used for tasks that need to run periodically (like a "Heartbeat" check for a microservice).**

---

5. Summary Scenario: The "Batch Logger"

If you were building a system to process logs:

1. **Use an ExecutorService to manage a pool of threads so you don't overwhelm the CPU.**
2. **Each log-saving task is a Runnable.**
3. **If multiple threads are updating a shared "Total Logs Processed" counter, you use an AtomicLong or a synchronized block to prevent a Race Condition.**

One-Day Roadmap: Final Technical Summary

Topic	Key "Senior" Insight
OOP/SOLID	Favor Composition over Inheritance; depend on Abstractions.
Collections	Use ArrayList for speed, ConcurrentHashMap for thread-safety.
JVM	Stack for references, Heap for objects. GC is non-deterministic.
Concurrency	Use ExecutorService instead of new Thread(). Avoid shared mutable state.